Secure Handover Protocol for V2X Networks

Secure bootstrap and Secrecy Forwarding handover with Intel SGX road-side units:
overview, strong and weak points, and step-by-step guide

Repository	github.com/IVNETGroup7/Secure-Handover-Protocol-For-V2X-Networks
Clone (HTTPS)	`https://github.com/IVNETGroup7/Secure-Handover-Protocol-For-V2X-Networks.git`
Clone (SSH)	`git@github.com:IVNETGroup7/Secure-Handover-Protocol-For-V2X-Networks.git`
Group	IVNET Group 7
Date	19 October 2026

Written in C++17 with OpenSSL 3 and the Intel SGX SDK (simulation mode). Runs as separate processes on raw Ethernet sockets, inside an SGX enclave, and in an Artery + OMNeT++ + SUMO vehicular network simulation.

Contents

1. What this project is 3

2. Strong points 4

3. Weak points and what can get better 5

3.1 Validation . . . 5

3.2 Design and implementation . . . 5

4. How to run it, step by step 6

4.1 What you need . . . 6

4.2 Get the code . . . 6

4.3 First run: the protocol without SGX (about 5 minutes) . . . 6

4.4 Second run: the RSU inside SGX (about 30 minutes, mostly waiting) . . . 6

4.5 Third run: the Artery simulation (1-2 hours, mostly waiting) . . . 7

4.6 When something goes wrong . . . 8

1. What this project is

Cars talk to road-side units (RSUs) along the road. Before an RSU can trust a car, and before the car can trust the RSU, both need shared secret keys. This project implements a **secure bootstrap** protocol that creates those keys between a **vehicle**, an **RSU** and an **External Security Agent (ESA)**, and a **Secrecy Forwarding handover** that moves an already trusted vehicle to the next RSU on its way **without repeating the bootstrap and without asking the ESA again**.

The RSU is a **Trusted Execution Environment (TEE) gateway**: its secrets (keys, one-time passwords, the decryption of the vehicles' data) live inside an **Intel SGX enclave**, a protected area of memory that even the RSU's own operating system cannot read. After a vehicle is trusted, it sends **SmartData** (measurements such as location and sensor values); the enclave checks and decrypts it (Secure Marshaling) and forwards it, re-encrypted, to a cloud server.

The same protocol code runs in three settings:

Setting	What runs	Used for
Processes on raw sockets (repository root)	ESA, RSU and vehicle as 3 Linux processes in a Docker container; optional cloud	Checking the protocol message by message
SGX gateway (rsu-sgx-gateway/)	The RSU's secret part inside an SGX enclave; a self-test with two enclaves (two RSUs) in one process	Checking the enclave, the handover and the data path
Artery simulation (artery-sim/)	2 SGX RSUs and 1 car on a 1 km highway (OMNeT++ + SUMO), ITS-G5 radio	Checking the protocol over a realistic vehicular radio and mobility

2. Strong points

- **Secrets stay in the enclave.** K_dg, K_g, K_eg and the handover keys are created and used only inside SGX; the App sees ciphertexts and public values. The enclave never prints a secret.
- **Fast handover by pre-computation.** The target RSU prepares K_dh, KT_i and auth before the vehicle arrives; the handover itself is two short messages (0.68 ms in the simulation), with no ESA and no new bootstrap.
- **Freshness and replay protection.** Time-slot OTPs (± 100 ms), the one-time pre-computed handover, the echoed OTP and auth' = SHA256(auth) in Forwarding_Granted.
- **Authenticated encryption everywhere** (AES-GCM) and constant-time comparisons.
- **Reboot recovery.** Sealed state per enclave restores the active sessions (with Y_V and t_i, so a vehicle can still be handed over after a restart).
- **The same code in three environments** (sockets, SGX, Artery) and one source for the handover equations, which removes a whole class of vehicle/RSU mismatches.
- **Negative tests** in the self-test: tampering, late messages, unknown identities, replays, old keys.
- **Reproducible setup.** setup-vm.sh and setup_artery.sh are idempotent and install everything from scratch; builds have no compiler warnings (-Wall -Wextra).

3. Weak points and what can get better

3.1 Validation

Weak point	How to improve it
One Artery run per configuration, one vehicle, one seed: no statistics	Repeat with several seeds (repeat = 30 in omnetpp.ini) and report means with confidence intervals
D500m_5G not run yet; it is 802.11p at 631 mW, not a real 5G NR model	Run it; for real 5G use Simu5G or label it explicitly as a power/range variant
No traffic load: a single car on an empty channel	Many vehicles, other services (CAM/DENM), several densities; measure channel busy ratio and DCC states
Handover delay excludes the pre-announcement and the backhaul	Also measure the service interruption: the gap between the last packet accepted by rsu[0] and the first by rsu[1]
No comparison baseline	Compare against a full re-bootstrap with rsu[1] (and with no TEE) for delay, messages and bytes
Fixed handover point (x = 500 m) and RSU spacing	Sweep handoverX, the RSU distance and the speed; trigger the handover on signal strength instead of a coordinate
No attacks in the simulation (only in the self-test)	Add attacker nodes: replay, forged Handover_Command, jamming, a vehicle that never arrives
SGX simulation mode only	Measure ECALL latency on SGX hardware (SGX_MODE=HW) and add remote attestation
Scenario tests, no unit tests or CI	Unit tests for Crypto, Handover and the codecs; fuzz the parsers; GitHub Actions running make and the self-test (SIM mode)
Security argued informally	Model the handover in ProVerif or Tamarin (secrecy of K_dh, authentication, replay)

3.2 Design and implementation

- **τ leaves the source enclave.** It is handed to the untrusted App, which sends it over the backhaul. It cannot forge the handover alone (an ECDH private key is still needed), but it should travel encrypted enclave-to-enclave.
- **Handover_Command is not authenticated.** A forged one cannot complete a handover, but it can make the vehicle waste a request; it could be signed by the source enclave.
- **The source RSU keeps the session** after the handover (no revocation message), and the optional pre-load of the next RSU's key (Y_G_i+3) is not implemented.
- **The vehicle prints its keys** in its trace (demonstration output) and sends the Activation_Request in place of the operator, so UUID and SHA256(f_w) cross the radio in clear.
- **PTP timestamps** are taken when a message is handed to the stack, not when the radio transmits it; network queues (DCC) add error.
- **The plain RSU** (BootstrapService in the RSU role) has no handover; only the SGX gateway does.
- **One OTP window** (± 1 slot of 100 ms) for everything: radio queuing delays can push valid packets outside it.
- The Artery build was verified on the Ubuntu 24.04 VM only; the Arch Linux and opp_env paths of setup_artery.sh were not tested.

4. How to run it, step by step

> This section assumes no previous experience. Follow the steps in order and copy the commands exactly. Everything is typed in a **terminal**: a window where you type commands and press **Enter** to run them.

4.1 What you need

- A computer with **Ubuntu 24.04**, connected to the internet. (The commands below are for Ubuntu; setup-vm.sh itself also supports Fedora and Arch Linux.)
- **8 GB of memory** or more, and **up to 20 GB of free disk space** (the VM disk is capped at 20 GB; in practice it uses about 10 GB).
- A processor with **virtualization enabled** (Intel VT-x or AMD-V; it is usually on; if step 4.4 says KVM is missing, enable it in the computer's BIOS/UEFI settings).
- Your user password (the computer asks for it when installing software).
- Time: about 30 minutes for parts 4.2-4.4, 1-2 hours more for the simulation (4.5), mostly waiting.

4.2 Get the code

1. Download the project. Use the **HTTPS** link (works for everyone):

```
cd ~
git clone https://github.com/IVNETGroup7/Secure-Handover-Protocol-For-V2X-Networks.git
```

If you have an SSH key registered on your GitHub account you can use the SSH link instead:

```
git clone git@github.com:IVNETGroup7/Secure-Handover-Protocol-For-V2X-Networks.git
```

2. Enter the project folder (every later command of 4.3 and 4.4 starts from here):

```
cd ~/Secure-Handover-Protocol-For-V2X-Networks
```

4.3 First run: the protocol without SGX (about 5 minutes)

1. Allow your user to use Docker, then **log out of the computer and log in again** (or restart it):

```
sudo usermod -aG docker $USER
```

2. Open a terminal again, enter the folder and run the three nodes (vehicle, RSU, ESA):

```
cd ~/Secure-Handover-Protocol-For-V2X-Networks
make pretty
```

> **Success looks like:** many lines describing each protocol step, ending with `bootstrap OK` (`exit codes: vehicle 0, rsu 0, esa 0`).

4.4 Second run: the RSU inside SGX (about 30 minutes, mostly waiting)

SGX needs a special software kit, which is installed inside a **virtual machine** (VM): a computer simulated inside your computer, so nothing on your own system has to change.

1. Create an SSH key (it lets your computer enter the VM without a password). Press **Enter** three times when it asks questions. If it says the file already exists, type **n** and continue.

```
ssh-keygen -t ed25519
```

2. Create the VM. This installs the virtualization software, downloads Ubuntu and installs the SGX kit inside the VM. It takes 10-25 minutes:

```
./setup-vm.sh
```

> **Success looks like:** `=== Success! Environment configured and VM 'ubuntu24-tee-vm' is running ===`. Then **log out and log in again** once (so your user may control VMs).

3. Build and test the SGX gateway. The command copies the project into the VM (to the folder ~/v2x there), builds it and runs the self-test:

```
cd ~/Secure-Handover-Protocol-For-V2X-Networks/rsu-sgx-gateway
make run
```

> **Success looks like:** a results table with six lines ending in **ok** (including `4. handover rsu[0] -> rsu[1] ... Secrecy Forwarding (4h-4m):` ok) and the last line `gateway OK`.

4. Optional: the same RSU on the network with a cloud server (it may ask for your password once):

```
make network
```

4.5 Third run: the Artery simulation (1-2 hours, mostly waiting)

The simulation is built and run **inside the VM**, because that is where the SGX kit is.

1. Make sure the VM has the latest copy of the project (step 4.4.3 does it; run it again whenever you change the code on your computer):

```
make -C ~/Secure-Handover-Protocol-For-V2X-Networks/rsu-sgx-gateway
```

2. Find the VM's network address and keep it in the variable `VM` (the second line prints it, e.g. 192.168.122.69), then enter the VM. The prompt changes to `ubuntu@ubuntu24-tee-vm`: from now on the commands run inside the VM. Type **exit** to come back to your computer. If it prints nothing or says domain is not running, start the VM first with `virsh -c qemu:///system start ubuntu24-tee-vm`, wait one minute and repeat this step.

```
VM=$(virsh -c qemu:///system domifaddr ubuntu24-tee-vm \
     | awk '/ipv4/ {split($4, a, "/"); print a[1]}')
echo $VM
ssh ubuntu@$VM
```

3. Install OMNeT++, Artery and SUMO and build the scenario. **Do not close the window** while it runs; if it stops (for example the connection drops), run the same command again: it continues where it stopped.

```
cd ~/v2x/artery-sim
./setup_artery.sh --install-sgx
```

> **Success looks like:** `===` done `===` followed by the two commands to run the simulation.

4. Run the simulation (first configuration, 802.11p at 200 mW):

```
cd ~/v2x/artery-sim/deps/artery/build
cmake -DRUN_FLAGS="-u Cmdenv -c D250m_80211p" .
cmake --build . --target run_tee_v2x_handover
```

> **Success looks like (about one minute):** `node[0] -> vehicle ready (with rsu[0])` near `t` = 0.64 s, `rsu[1] -> handover done (node[0] now with rsu[1])` near `t` = 18.14 s, and finally `Simulation time limit reached -- at t=40s`.

4.6 When something goes wrong

Message	What to do
permission denied ... docker.sock	You skipped the log out / log in after usermod (4.3.1).
network 'default' is not active	sudo virsh net-start default && sudo virsh net-autostart default
Host key verification failed (ssh)	ssh-keygen -R $VM (the VM's address, step 4.5.2), then try again and answer yes
configure: error: Before running 'configure', please use 'source setenv'	The VM has an old copy of the project: run step 4.5.1 on your computer, then 4.5.3 again.
Could NOT find Python (missing: Python_INCLUDE_DIRS ...)	Old copy of the project (same fix), or install it by hand in the VM: sudo apt install -y python3-dev
Cannot load library ... undefined symbol	A build was interrupted and left empty files. In the VM: cd ~/v2x/artery-sim/deps/artery/build, then find . -name '*.o' -size 0 -delete, then cmake --build . -j 2
tee_v2x_handover skipped: no Intel SGX SDK	Run setup_artery.sh with --install-sgx (inside the VM).
The VM does not answer (ssh timeout)	virsh -c qemu:///system start ubuntu24-tee-vm, wait one minute, try again

Do not run two builds of the simulation at the same time (for example two terminals running setup_artery.sh): they overwrite each other's files.